

Migrate Foundry Hosted Agents from initial → refreshed public preview

Author: Linda Zhang, AI APP GBB

A step-by-step playbook based on migrating this repo on **May 5, 2026** (17 days before the May 22, 2026 retirement of the initial preview backend). Reference: [Migrate hosted agents to the refreshed public preview](#).

0. Prerequisites

- `azd ≥ 1.23.0` — upgrade if older: `powershell powershell -ex AllSigned -c "Invoke-RestMethod 'https://aka.ms/install-azd.ps1' | Invoke-Expression"`
Then refresh PATH (close+reopen terminal or): `powershell $env:Path = [Environment]::GetEnvironmentVariable('Path','Machine') + ';' + [Environment]::GetEnvironmentVariable('Path','User')` `azd version # expect ≥ 1.23`
 - Install the Foundry agents extension: `powershell azd ext install azure.ai.agents`
 - Sign in to the right tenant/subscription: `powershell azd auth login --tenant-id <TENANT_ID>` `az login --tenant <TENANT_ID>` `az account set --subscription <SUBSCRIPTION_ID>`
 - `azure-ai-projects ≥ 2.1.0` (in the venv used by `scripts/deploy_all.py`):
`powershell pip install --upgrade "azure-ai-projects>=2.1.0" pyyaml azure-identity`
-

1. Update Python packages

`shared/requirements-base.txt`

Replace the framework-adapter package with the protocol library:

```
- azure-ai-agentserver-agentframework==1.0.0b16 - azure-ai-agentserver-core==1.0.0b16
+ azure-ai-agentserver-responses==1.0.0b5 + azure-ai-agentserver-core>=2.0.0b3 +
azure-ai-projects>=2.1.0
```

The migration article mentions `2.0.0b1` for `azure-ai-agentserver-responses`, but the version actually published to PyPI as of this migration is `1.0.0b5` (which depends on `azure-ai-agentserver-core>=2.0.0b3`). Check PyPI for the latest before pinning.

`shared/requirements-af.txt`

```
- agent-framework-core==1.0.0rc6 - agent-framework-foundry==1.0.0rc6 -
agent-framework-openai==1.0.0rc6 + agent-framework-core==1.2.2 +
agent-framework-foundry==1.2.2 + agent-framework-openai==1.2.2 +
```

```
agent-framework-orchestrations==1.0.0b260429 +
agent-framework-foundry-hosting==1.0.0a260429
```

The new `agent-framework-foundry-hosting` package is the bridge between Agent Framework and the protocol library.

2. Replace the agent entry point

`shared/agent_host.py`

```
- from azure.ai.agentserver.agentframework import from_agent_framework + from
agent_framework_foundry_hosting import ResponsesHostServer from agent_framework import
Agent from agent_framework.foundry import FoundryChatClient def build_agent(name,
instructions, tools=None) -> Agent: return Agent( client=_client(), name=name,
instructions=instructions, tools=list(tools or []), + default_options={"store":
False}, # platform manages history now ) - async def serve(agent: Agent) -> None: -
await from_agent_framework(agent).run_async() + def serve(agent: Agent) -> None: +
ResponsesHostServer(agent).run()
```

Each agents/`<name>/main.py`

`serve()` is now sync, so drop the `asyncio` wrapper:

```
- import asyncio ... - async def main() -> None: + def main() -> None: agent =
build_agent(...) - await serve(agent) + serve(agent) if __name__ == "__main__": -
asyncio.run(main()) + main()
```

3. Switch local tools to `@tool`

The `@ai_function` decorator is replaced by `@tool(approval_mode=...)`:

```
+ from agent_framework import tool + @tool(approval_mode="never_require") def
inventory_check( product_ids: Annotated[list[str], Field(description="...")], ) ->
dict: ...
```

Apply to every function in `shared/zava_tools.py` and `shared/handoff.py`.

Use `approval_mode="never_require"` for tools that must run without user confirmation (everything in this repo).

4. Update protocol versions and deploy script

`scripts/deploy_all.py`

Confirm two things:

```
ProtocolVersionRecord(protocol=..., version="1.0.0") # not "v1" AIProjectClient(
endpoint=project_endpoint, credential=DefaultAzureCredential(), allow_preview=True, #
required to use agent_name routing )
```

Remove any `tools=[...]` from `HostedAgentDefinition` — tools are no longer defined at version-create time.

Invocation code (already correct in this repo)

```
client = project.get_openai_client(agent_name="cora") # not extra_body resp =
client.responses.create(input="...")
```

5. Make azd postdeploy use your venv (Windows)

Edit `azure.yaml` so the hook runs the right Python:

```
hooks: postdeploy: shell: pwsh run: | $py = if (Test-Path ../.venv/Scripts/python.exe)
{ '../.venv/Scripts/python.exe' } else { 'python' } & $py ../scripts/deploy_all.py
```

6. Build new images and register new versions

Force a fresh tag so ACR rebuilds (otherwise it skips when the SHA matches):

```
$env:AZURE_BUILD_TAG = "refresh-" + (Get-Date -Format "yyyyMMddHHmmss") azd hooks run
postdeploy
```

Each agent should print `registered <name>@<n>`.

Confirm each new version reaches active:

```
python ../scripts/check_status.py
```

7. Cut traffic over to the new versions

This step is critical. Creating a new version does not move traffic. You must PATCH each agent's `version_selector.version_selection_rules` to route 100% to the new version:

```
PATCH /agents/{name}?api-version=2025-11-15-preview { "agent_endpoint": {
"version_selector": { "version_selection_rules": [ {"type": "FixedRatio",
"agent_version": "<new>", "traffic_percentage": 100} ] } } }
```

This repo automates it via [scripts/update_routing.py](#):

```
python ./scripts/update_routing.py
```

Verify by sending a request and inspecting `agent_reference.version` in the response — it must equal your new version, and the legacy `metadata.foundry_agents_metadata.package` field should be `null`/absent (it was `azure-ai-agentserver-agentframework` on the old backend).

8. Re-grant RBAC to per-agent identities

In the refreshed preview, each agent gets its **own dedicated Entra identity** (see `instance_identity.principal_id` on each version). The project managed identity is no longer the runtime identity — RBAC roles you previously gave to the project MI do **not** transfer.

For this repo, every agent calls the gpt-4o model and Cora calls sibling agents, so each needs **Azure AI User** at the project scope:

```
python ./scripts/assign_agent_rbac.py
```

(The script is in this repo; it discovers each agent's `principal_id` from the versions list and creates the role assignment.)

If your tools touch other Azure resources (Cosmos DB, Storage, Key Vault, ...), grant the appropriate roles on those resources to each agent's principal too.

9. Smoke test

```
$env:AZURE_AI_PROJECT_ENDPOINT =  
"https://<account>.cognitiveservices.azure.com/api/projects/<project>" python  
scripts/smoke_test.py cora "hi briefly what can you help me with"
```

Look for in the response:

- `agent_reference.version` = your new version (✓ refreshed preview)
- new `agent_session_id` field
- response id starts with `caresp_` (refreshed preview format)
- `metadata` is `null` or doesn't reference `azure-ai-agentserver-agentframework`

Then test a handoff path:

```
python scripts/smoke_test.py cora "I want to redo my living room in emerald and brass"
```

Look for an `output[]` entry with type: `"function_call"` and name: `"handoff_to_specialist"`, then a final assistant message.

Migration checklist (this repo)

- [x] `azd ≥ 1.23.0` installed
- [x] `azd ext install azure.ai.agents`
- [x] `azure-ai-projects ≥ 2.1.0`
- [x] `azure-ai-agentserver-agentframework` removed
- [x] `azure-ai-agentserver-responses + agent-framework-foundry-hosting` added
- [x] `from_agent_framework().run() → ResponsesHostServer(...).run()`
- [x] `default_options={"store": False}` set on Agent
- [x] Local tools decorated with `@tool(approval_mode="never_require")`
- [x] Protocol version literal `"v1" → "1.0.0"`
- [x] `tools=[...]` removed from `HostedAgentDefinition`
- [x] `extra_body={"agent_reference": ...} → get_openai_client(agent_name=...)`
- [x] `AIProjectClient(allow_preview=True)`
- [x] Container images rebuilt and pushed
- [x] New versions reach status `active`
- [x] Endpoint routing patched to direct 100% to new versions
- [x] Per-agent dedicated identities granted Azure AI User at project scope
- [x] Smoke test confirms new backend (`response shape + agent_reference.version`)

Pitfalls hit during this migration

1. **azd was 1.12.0** — the `ext` subcommand didn't exist; required upgrade.
2. **Wrong package version** — used `azure-ai-agentserver-responses==2.0.0b1` (from the doc) but PyPI only had up to `1.0.0b5`. Check actual published versions before pinning.
3. **ACR build skipped** because `_image_exists` matched the previous git SHA tag (uncommitted code edits don't change the SHA). Fix: set `$env:AZURE_BUILD_TAG` to a unique value before running the postdeploy hook.
4. **postdeploy hook ran system Python** without `pyyaml` — fixed by using the venv's interpreter.
5. **New version was active but traffic still went to the old one.** The refreshed preview requires an explicit `PATCH /agents/{name}` to update `version_selector`. We discovered this only because the smoke-test response metadata still showed `azure-ai-agentserver-agentframework==1.0.0b16`.